

Trabajo Práctico N.º 3: Primer proyecto en RTOS

El primer proyecto tiene como objetivo principal realizar un esqueleto de un proyecto funcional en FreeRTOS, que luego podrá ser utilizado como *template* para próximos proyectos.

Se implementará una tarea simple que realizará el parpadeo del LED de la placa BluePill.

Para ello será necesario:

1. Crear una carpeta para el proyecto junto con una estructura interna de subcarpetas.
2. Descargar la librería CMSIS y copiar los archivos necesarios a la carpeta del proyecto.
3. Descargar FreeRTOS y copiar los archivos necesarios a la carpeta del proyecto.
4. Modificar algunos archivos de CMSIS y FreeRTOS para vincular ambos proyectos.
5. Generar desde cero una serie de archivos necesarios para el funcionamiento de la aplicación.
6. Compilar y grabar el programa en la placa BluePill.

Paso 1: Estructura de subcarpetas

La carpeta principal de nuestra aplicación se denominará `rtos-blink`. La estructura general del proyecto será:

```
rtos-blink/  
├── CMSIS/  
├── FreeRTOS/  
├── src/  
└── ld/
```

- **rtos-blink:** nombre de la carpeta principal del proyecto.
- **FreeRTOS:** carpeta con los archivos del sistema operativo.
- **CMSIS:** carpeta con los archivos de las librerías CMSIS.
- **src:** carpeta con los archivos fuente exclusivos de la aplicación (`main.c`, por ejemplo).
- **ld:** carpeta que contiene el *linker script*.

Paso 2: Common Microcontroller Software Interface Standard (CMSIS)

CMSIS es una librería promovida por ARM que define registros y funciones para el manejo del procesador. Permite estandarizar estas funciones y facilita la migración entre distintos procesadores Cortex-M.

Estructura del proyecto CMSIS

CMSIS está compuesto por distintos módulos:

■ CMSIS-Core

- Define registros y funciones del procesador: NVIC, SysTick, control de interrupciones, etc.
- Incluye archivos fundamentales para iniciar una aplicación, como `core_cm3.h` y `system_stm32f1xx.h`.
- Permite que el código sea portable entre distintos fabricantes (ST, NXP, TI, etc.).

■ CMSIS-Driver

- Define una API estándar para periféricos comunes como I²C, SPI y USART.
- Posee *templates* para que cada fabricante implemente la API para su hardware.

■ CMSIS-DSP

- Librería optimizada para procesamiento digital de señales: FFT, filtros, PID, etc.

■ CMSIS-RTOS

- API estándar para sistemas operativos en tiempo real, por ejemplo FreeRTOS y RTX.
- Permite escribir aplicaciones independientes del RTOS utilizado.
- En este proyecto no se utilizará para no agregar complejidad innecesaria.

En la librería CMSIS se encuentran las implementaciones generales relativamente independientes del hardware. A partir de ellas, cada fabricante proporciona la implementación particular para sus microcontroladores.

Archivos CMSIS-Core necesarios

La librería CMSIS puede descargarse desde:

<https://www.arm.com/technologies/cmsis>

Dentro del proyecto CMSIS se debe acceder a:

`CMSIS_6/CMSIS/Core/Include`

y copiar los archivos necesarios a la carpeta **CMSIS** del proyecto.

La estructura utilizada será:

```
CMSIS/  
├── cmsis_compiler.h  
├── cmsis_gcc.h  
├── cmsis_version.h  
├── core_cm3.h  
├── m-profile/  
│   └── cmsis_gcc_m.h
```

Archivos específicos del STM32F1

Los archivos específicos del microcontrolador utilizado en la BluePill se obtienen del proyecto **cmsis-device-f1** de STMicroelectronics:

<https://github.com/STMicroelectronics/cmsis-device-f1>

De **cmsis-device-f1/Source/Templates** se copian:

- **system_stm32f1xx.c**
- **system_stm32f1xx.h**

El archivo **system_stm32f1xx.c** incluye la función **SystemInit()**, encargada de inicializar el reloj, configurar la memoria Flash y otros parámetros básicos. También contiene la función **SystemCoreClockUpdate** que debe ser llamada desde **main()** para actualizar la variable global **SystemCoreClock**, donde se almacena la frecuencia de la CPU.

De **cmsis-device-f1/Source/Templates/gcc** se copia:

- **startup_stm32f102xb.s**

Este archivo está escrito en assembler y contiene la tabla de vectores de interrupción. El primer vector corresponde al Reset y es atendido por la función **Reset_Handler**.

La función **Reset_Handler** realiza, entre otras, las siguientes operaciones:

- Llama a **SystemInit()**.
- Copia la sección **.data** desde Flash a RAM.
- Inicializa en cero la sección **.bss**.
- Llama a **__libc_init_array()** para realizar inicializaciones necesarias de **newlib**.
- Llama a **main()**.

De `cmsis-device-f1/Include` se copian:

- `stm32f1xx.h`
- `system_stm32f1xx.h`
- `stm32f103xb.h`

Paso 3: Linker Script

En la carpeta `ld` se copia:

`STM32F103XB_FLASH.ld`

desde:

`cmsis-device-f1/Source/Templates/gcc/linker`

El *linker script* indica al enlazador cómo deben ubicarse las distintas secciones de los archivos objeto en la memoria del microcontrolador.

Entry Point y stack

```
/* Entry Point */
ENTRY(Reset_Handler)

/* Highest address of the user mode stack */
_estack = 0x20004FFF; /* end of RAM */
```

`ENTRY(Reset_Handler)` indica el nombre de la primera función que se ejecutará en el sistema.

La variable `_estack` recibe la dirección más alta de la RAM y luego es utilizada por el archivo de *startup* para cargar el MSP (*Main Stack Pointer*).

Tamaños mínimos de Heap y Stack

```
/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;
_Min_Stack_Size = 0x400;
```

Estos símbolos establecen tamaños mínimos para el *heap* y el *stack*. No representan necesariamente su tamaño final, sino que son usados posteriormente para verificar que la memoria requerida no exceda la RAM disponible.

- `_Min_Heap_Size = 0x200`: reserva un mínimo de 512 bytes.
- `_Min_Stack_Size = 0x400`: reserva un mínimo de 1024 bytes.

Definición de las áreas de memoria

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
    RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

Este bloque describe el mapa físico de memoria:

- `FLASH (rx)`: memoria Flash de lectura y ejecución.
- `RAM (xrw)`: memoria RAM de lectura, escritura y ejecución.

Definición y ubicación de las secciones

El bloque `SECTIONS` organiza el contenido del programa dentro de la memoria.

Tabla de vectores de interrupción

```
.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector))
    . = ALIGN(4);
} >FLASH
```

La sección `.isr_vector` contiene la tabla de vectores y debe ubicarse al principio de la memoria Flash. La directiva `KEEP()` impide que el enlazador elimine esta sección aunque no detecte referencias explícitas hacia ella.

Código y datos de solo lectura

```
.text :
{
    . = ALIGN(4);
```

```
*(.text)
*(.text*)
. = ALIGN(4);
_etext = .;
} >FLASH

.rodata :
{
. = ALIGN(4);
*(.rodata)
*(.rodata*)
. = ALIGN(4);
} >FLASH
```

- `.text`: agrupa el código ejecutable y lo ubica en Flash.
- `.rodata`: agrupa constantes y cadenas de solo lectura.

Variables inicializadas: sección `.data`

```
_sidata = LOADADDR(.data);

.data :
{
. = ALIGN(4);
_sdata = .;
*(.data)
*(.data*)
. = ALIGN(4);
_edata = .;
} >RAM AT> FLASH
```

La sección `.data` contiene variables globales y estáticas inicializadas. Durante la ejecución se ubican en RAM, pero sus valores iniciales se almacenan en Flash.

El `Reset_Handler` utiliza:

- `_sidata`: dirección en Flash donde comienzan los valores iniciales.
- `_sdata`: dirección en RAM donde deben copiarse.
- `_edata`: dirección final de la sección en RAM.

Variables no inicializadas: sección .bss La sección `.bss` contiene variables globales y estáticas no inicializadas. Al inicio del programa, el `Reset_Handler` pone a cero esta región de RAM.

Paso 4: Incorporación de FreeRTOS

FreeRTOS es un sistema operativo de tiempo real utilizado en microcontroladores y pequeños microprocesadores.

Los archivos del proyecto pueden descargarse desde:

<https://www.freertos.org/>

Los archivos necesarios se encuentran en:

`FreeRTOSxx/FreeRTOS-LTS/FreeRTOS/FreeRTOS-Kernel`

donde `FreeRTOSxx` representa la versión descargada.

Los archivos del kernel y las carpetas `examples` e `include` pueden copiarse a la carpeta `FreeRTOS` del proyecto.

Dentro de `portable` se deben conservar únicamente las implementaciones necesarias para nuestra plataforma:

- `portable/GCC/ARM_CM3`: port específico para Cortex-M3 usando GCC.
- `portable/MemMang`: implementaciones de gestión de memoria dinámica.

La estructura resultante será:

```
FreeRTOS/
├── examples/
├── include/
└── portable/
    ├── GCC/
    │   └── ARM_CM3/
    └── MemMang/
```

Gestión de memoria: `heap_4.c`

Dentro de `portable/MemMang` existen diferentes archivos `heap_x.c`, cada uno con una estrategia de gestión de memoria dinámica.

Para este proyecto se utilizará:

FreeRTOS/portable/MemMang/heap_4.c

heap_4.c es un gestor de propósito general que permite reutilizar memoria liberada y agrupar bloques adyacentes. Debe agregarse a la variable SRCS del Makefile.

Paso 5: Archivos a modificar

Una vez incorporados CMSIS y FreeRTOS es necesario realizar algunas modificaciones para vincular el proyecto basado en CMSIS con FreeRTOS.

1. system_stm32f1xx.c

La función SystemInit() debe configurar el reloj del microcontrolador a 72 MHz.

```
void SystemInit(void)
{
    /* 1. Habilitar HSE */
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY));

    /* 2. Configurar Flash: Prefetch + 2 wait states */
    FLASH->ACR |= FLASH_ACR_PRFTBE;
    FLASH->ACR &= ~FLASH_ACR_LATENCY;
    FLASH->ACR |= FLASH_ACR_LATENCY_2;

    /* 3. Configurar prescalers */
    RCC->CFGR |= RCC_CFGR_HPRE_DIV1; // AHB = SYSCLK /1
    RCC->CFGR |= RCC_CFGR_PPRE1_DIV2; // APB1 = SYSCLK /2
    RCC->CFGR |= RCC_CFGR_PPRE2_DIV1; // APB2 = SYSCLK /1

    /* 4. Configurar PLL (HSE * 9 = 72 MHz) */
    RCC->CFGR &= ~(RCC_CFGR_PLLSRC |
                  RCC_CFGR_PLLXTPRE |
                  RCC_CFGR_PLLMULL);
    RCC->CFGR |= (RCC_CFGR_PLLSRC | RCC_CFGR_PLLMULL9);

    /* 5. Encender PLL y esperar */
    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY));

    /* 6. Cambiar SYSCLK a PLL */
    RCC->CFGR &= ~RCC_CFGR_SW;
```



```
RCC->CFGR |= RCC_CFGR_SW_PLL;
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);

/* 7. Actualizar variable global */
SystemCoreClock = 72000000;
}
```

2. port.c

El archivo `port.c` contiene las funciones que utiliza FreeRTOS para vincularse con el hardware. En nuestro caso corresponde al port `ARM_CM3` para Cortex-M3.

Al final del archivo se agregan:

```
void SVC_Handler(void)
    __attribute__((alias("vPortSVCHandler")));

void PendSV_Handler(void)
    __attribute__((alias("xPortPendSVHandler")));

void SysTick_Handler(void)
    __attribute__((alias("xPortSysTickHandler")));
```

Estas líneas vinculan los nombres de los manejadores definidos según CMSIS con las funciones proporcionadas por FreeRTOS.

3. syscalls.c

El archivo `syscalls.c` no es requerido directamente por FreeRTOS, sino por la librería `newlib`, que implementa funciones como `printf()`, `scanf()` y `malloc()`.

Puede considerarse un archivo genérico, ya que normalmente puede reutilizarse entre aplicaciones sin modificaciones.

```
#include <sys/stat.h>
#include <errno.h>
#include <stdint.h>

void _exit(int status)
{
    (void)status;
    while (1) {}
}
```

```
}

int _close(int file) { return -1; }

int _fstat(int file, struct stat *st)
{
    st->st_mode = S_IFCHR;
    return 0;
}

int _isatty(int file) { return 1; }
int _lseek(int file, int ptr, int dir) { return 0; }
int _read(int file, char *ptr, int len) { return 0; }
int _write(int file, char *ptr, int len) { return len; }

void _init(void) {}
```

Paso 6: Archivos a agregar

Los archivos que deben generarse desde cero son los relacionados con la aplicación y con la configuración del sistema:

- `main.c`: contiene la inicialización del hardware, la creación de las tareas y el arranque del scheduler.
- `Makefile`: controla la compilación y el enlace del proyecto.
- `FreeRTOSConfig.h`: contiene las macros `#define` que configuran el kernel de FreeRTOS.

Paso 7: Makefile

Un Makefile contiene las reglas necesarias para automatizar la construcción de la aplicación.

En una aplicación embebida permite:

1. Definir los archivos fuente `.c` y `.s`.
2. Configurar el compilador GCC.
3. Controlar el enlace, indicando el *linker script* y los objetos.
4. Generar archivos `.elf`, `.bin`, `.hex` y archivos de informe como `.map`.

5. Administrar dependencias.
6. Automatizar comandos como `make clean` y `make flash`.

Variables del toolchain

```
PREFIX = arm-none-eabi -  
CC = $(PREFIX)gcc  
OBJCOPY = $(PREFIX)objcopy
```

- PREFIX: prefijo del toolchain ARM.
- CC: compilador C, utilizado también para enlazar y ensamblar.
- OBJCOPY: herramienta para convertir el archivo `.elf` a `.bin` o `.hex`.

Banderas de compilación

La variable `CFLAGS` incluye, entre otras:

- `-mcpu=cortex-m3`: selecciona Cortex-M3 como CPU destino.
- `-g`: incorpora información de depuración.
- `-I<dir>`: agrega directorios de búsqueda de archivos de cabecera.
- `-DSTM32F103xB`: define el microcontrolador para CMSIS.
- `-Wall`: habilita advertencias comunes.
- `-ffunction-sections` y `-fdata-sections`: crean secciones independientes para funciones y variables.
- `-MMD -MP`: genera archivos de dependencias `.d`.

Flags de enlace

La variable `LDFLAGS` contiene:

- `-T ld/STM32F103XB_FLASH.ld`: selecciona el linker script.
- `-nostartfiles`: evita los archivos de arranque predeterminados del toolchain.
- `-Wl,-gc-sections`: elimina secciones no utilizadas.
- `-Wl,-Map=app.map`: genera el mapa de memoria.

Las opciones que comienzan con `-Wl`, son transmitidas por GCC directamente al enlazador GNU `ld`.

Fuentes, objetos y dependencias

- **SRCS**: lista de todos los archivos fuente de la aplicación.
- **OBJS**: lista de archivos objeto generados a partir de **SRCS**.
- **DEPS**: dependencias automáticas generadas.

Reglas principales

```
all: app.elf app.bin app.hex

app.elf: $(OBJS)
        $(CC) $(CFLAGS) $^ -o $@ $(LDFLAGS)

app.bin: app.elf
        $(OBJCOPY) -O binary $< $@

app.hex: app.elf
        $(OBJCOPY) -O ihex $< $@
```

La regla `all` genera los tres formatos principales. El archivo `app.elf` es el utilizado por OpenOCD para grabar el microcontrolador.

Reglas de compilación

```
%.o: %.c
        $(CC) $(CFLAGS) -c -o $@ $<

%.o: %.s
        $(CC) -c -mcpu=cortex-m3 -mthumb -o $@ $<
```

Estas reglas utilizan el comodín `%` para evitar definir individualmente cada archivo objeto.

Dependencias automáticas

```
-include $(DEPS)
```

Esto permite recompilar únicamente los archivos afectados cuando cambia una cabecera.

Reglas auxiliares

```
flash: app.elf
    openocd -f interface/stlink.cfg \
    -f target/stm32f1x.cfg \
    -c "program app.elf verify reset exit"

clean:
    rm -f $(OBJS) $(DEPS) app.elf app.map
```

`make flash` programa la placa mediante OpenOCD y ST-Link. `make clean` elimina objetos, dependencias y archivos generados.

Paso 8: main.c

El archivo `main.c` es el corazón de la aplicación. Su función es inicializar el hardware, crear las tareas que se ejecutarán y, finalmente, ceder el control al scheduler de FreeRTOS.

```
#include "stm32f1xx.h"
#include "FreeRTOS.h"
#include "task.h"

void vBlinkTask(void *pvParameters)
{
    (void)pvParameters;

    while(1)
    {
        GPIOC->BSRR = GPIO_BSRR_BR13; // LED ON
        vTaskDelay(pdMS_TO_TICKS(500));

        GPIOC->BSRR = GPIO_BSRR_BS13; // LED OFF
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

int main(void)
{
    SystemCoreClockUpdate();
```

```
// Habilitar reloj GPIOC
RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;

// Configurar PC13 como salida push-pull a 10 MHz
GPIOC->CRH &= ~GPIO_CRH_CNF13;
GPIOC->CRH |= GPIO_CRH_MODE13_0;

// Crear tarea Blink
// Stack: 256 palabras (1024 bytes), prioridad 1
BaseType_t res =
    xTaskCreate(vBlinkTask, "Blink", 256, NULL, 1, NULL);

if (res == pdPASS)
{
    // Arrancar scheduler
    vTaskStartScheduler();
}

// Nunca deberia llegar aqui
for (;;)
}
```

En una aplicación bare-metal clásica, el `main()` suele contener el *big-loop* o lazo principal de la aplicación.

En FreeRTOS, en cambio, el `main()` crea las tareas y luego llama a `vTaskStartScheduler()`, momento a partir del cual FreeRTOS toma el control de la planificación del microprocesador.

Paso 9: FreeRTOSConfig.h

Antes de compilar la aplicación es indispensable crear `FreeRTOSConfig.h`. Este archivo proporciona al kernel la información necesaria sobre el hardware y las funcionalidades que se desean habilitar.

Una buena práctica es ubicarlo en la carpeta `src/`.

El siguiente ejemplo está configurado para una BluePill funcionando a 72 MHz:

```
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/* ----- Seccion 1: Configuracion del Hardware ----- */
#define configCPU_CLOCK_HZ          ( 72000000UL )
```

```
#define configTICK_RATE_HZ          ((TickType_t)1000)

/* ----- Seccion 2: Configuracion del Kernel ----- */
#define configUSE_PREEMPTION        1
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configSUPPORT_STATIC_ALLOCATION 0
#define configTOTAL_HEAP_SIZE      ((size_t)(5*1024))
#define configMAX_TASK_NAME_LEN    ( 16 )
#define configUSE_16_BIT_TICKS     0

/* ----- Seccion 3: Hooks y Opciones de Depuracion ----- */
#define configUSE_IDLE_HOOK         0
#define configUSE_TICK_HOOK         0
#define configCHECK_FOR_STACK_OVERFLOW 0

/* ----- Seccion 4: Inclusion de APIs ----- */
#define INCLUDE_vTaskDelay          1

/* ----- Seccion 5: Prioridades de Interrupcion ----- */
/* En Cortex-M, prioridad 0 es la prioridad mas alta. */
#define configKERNEL_INTERRUPT_PRIORITY 255
#define configMAX_SYSCALL_INTERRUPT_PRIORITY 191

/*
 * Las interrupciones con prioridad numerica inferior a esta
 * no pueden llamar a la API de FreeRTOS.
 */

/* ----- Seccion 6: Mapeo de Handlers a CMSIS ----- */
#define vPortSVCHandler             SVC_Handler
#define xPortPendSVHandler          PendSV_Handler
#define xPortSysTickHandler         SysTick_Handler

#endif /* FREERTOS_CONFIG_H */
```

Parámetros principales

- configCPU_CLOCK_HZ: frecuencia de reloj de la CPU.
- configTICK_RATE_HZ: frecuencia del tick del RTOS. Con 1000 Hz se obtiene un tick cada 1 ms.
- configUSE_PREEMPTION: habilita la planificación preemptiva.

- `configTOTAL_HEAP_SIZE`: cantidad de memoria reservada para asignación dinámica del kernel.
- `INCLUDE_vTaskDelay`: habilita la función `vTaskDelay()`.
- `configKERNEL_INTERRUPT_PRIORITY`: prioridad utilizada por las interrupciones internas del kernel.
- `configMAX_SYSCALL_INTERRUPT_PRIORITY`: establece el límite de prioridad para las interrupciones que pueden utilizar la API de FreeRTOS.

Resultado esperado

Al compilar y grabar el proyecto, FreeRTOS debe iniciar correctamente el scheduler y ejecutar la tarea `vBlinkTask()`.

El LED conectado a PC13 deberá:

- encenderse durante 500 ms;
- apagarse durante 500 ms;
- repetir el ciclo indefinidamente.

La función `vTaskDelay()` no detiene el procesador: bloquea únicamente la tarea que la invoca, permitiendo que el scheduler ejecute otras tareas que se encuentren listas.